AULA 06

Matrizes

O vetor em duas dimensões · linhas e colunas · for aninhado

1	2	3
4	5	6
7	8	9

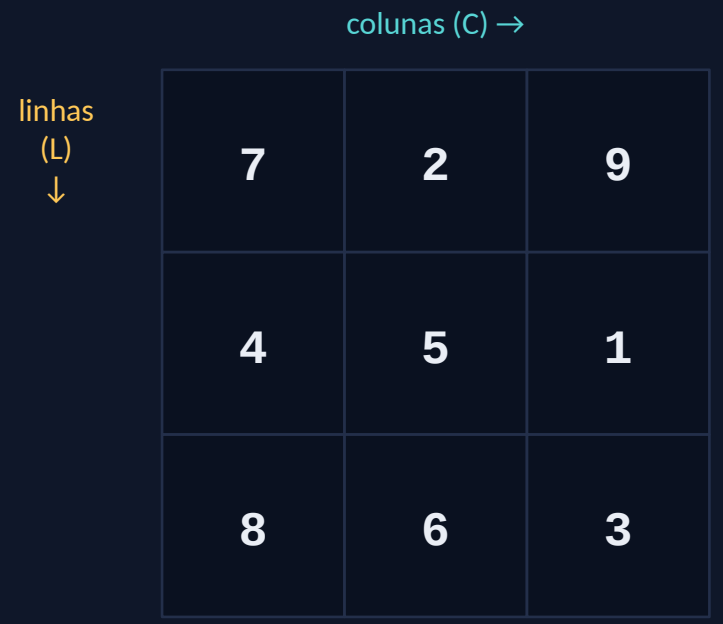
```c
#define LINHAS 3
#define COLUNAS 3
int A[LINHAS][COLUNAS];
```

Prof. Vinicius Breda

O que é uma matriz?

Uma matriz é um vetor de DUAS dimensões: uma tabela com linhas e colunas. Cada valor fica num cruzamento linha × coluna.

- Vetor = uma fila de valores (1 dimensão).
- Matriz = uma tabela de valores (2 dimensões).
- Todos os valores são do mesmo tipo.
- Acessamos cada valor por dois índices: [linha][coluna].



uma matriz 3 × 3 (3 linhas, 3 colunas)

Declaração e acesso: A[L][C]

Declaramos com dois tamanhos. O primeiro índice é a linha (L); o segundo, a coluna (C). Ambos começam em 0.

matriz.c

```c
#define LINHAS 3
#define COLUNAS 3

int A[LINHAS][COLUNAS];  // 3 x 3 = 9 valores
A[1][2] = 5;   // linha 1, coluna 2
```



`A[1][2]` = 1 (linha 1, coluna 2)

Cuidado: com 3 linhas e 3 colunas, os índices vão de 0 a 2. A[3][...] não existe.

Para que serve? Onde usamos matrizes?

Sempre que os dados formam uma TABELA (linhas e colunas):

Tabelas

notas dos alunos por bimestre

Imagens

os pixels de uma foto (grade)

Jogos

tabuleiro: xadrez, batalha naval, velha

Planilhas

linhas e colunas de dados

Mapas

grade de um cenário ou labirinto

Matemática

matrizes para somar, multiplicar

A regra: dados em linhas E colunas → matriz, percorrida com dois for (um dentro do outro).

Percorrendo com dois for (L e C)

Para visitar toda a matriz usamos dois for: o de fora anda nas LINHAS (L); o de dentro, nas COLUNAS (C).

percorrer.c

```
int L, C;
for (L = 0; L < LINHAS; L++) {   // linhas
  for (C = 0; C < COLUNAS; C++) { // colunas
    ... A[L][C] ...
  }
}
```

- Para cada linha L, o for de dentro visita TODAS as colunas C.
- A ordem: [0][0], [0][1], [0][2], [1][0], [1][1]...

	C=0	C=1	C=2
L=0	1	2	3
L=1	4	5	6
L=2	7	8	9

ordem em que o duplo for visita as posições

Matriz é passada por REFERÊNCIA

Como o vetor (Aula 05), a matriz vai por referência: o nome já é o endereço. A função altera a matriz original.

referencia.c

```c
void carregarMatriz(int M[LINHAS][COLUNAS]) {
    ...
}

carregarMatriz(A);   // sem & — o nome já é o endereço
```

- Não usamos & na chamada: carregarMatriz(A).
- A função mexe na matriz original (não numa cópia).
- Por isso carregarMatriz e somarMatriz conseguem preencher.
- Ao passar para a função, informe as COLUNAS no parâmetro: M[][COLUNAS].
- Com #define, escrevemos M[LINHAS][COLUNAS] e pronto.

Por que informar as colunas?
o C precisa saber o tamanho da coluna para achar cada linha na memória. A linha pode ficar em branco (M[][COLUNAS]), a coluna não.

Procedimento: carregarMatriz

Percorre a matriz com os dois for e lê (scanf) cada posição digitada pelo usuário.

carregar.c

```c
void carregarMatriz(int M[LINHAS][COLUNAS]) {
  int L, C;
  for (L = 0; L < LINHAS; L++) {
    for (C = 0; C < COLUNAS; C++) {
      printf("Elemento [%d][%d]: ", L, C);
      scanf("%d", &M[L][C]);
    }
  }
}
```

- L e C: os índices de linha e coluna.
- O for de fora anda nas linhas; o de dentro, nas colunas.
- scanf lê cada posição: &M[L][C].
- O & aponta o endereço daquela posição (como no vetor).
- A matriz original é preenchida (referência).

Procedimento: visualizarMatriz

Mostra a matriz em formato de tabela. O segredo é dar \n ao fim de cada LINHA.

visualizar.c

```c
void visualizarMatriz(int M[LINHAS][COLUNAS]) {
  int L, C;
  for (L = 0; L < LINHAS; L++) {
    for (C = 0; C < COLUNAS; C++) {
      printf("%d\t", M[L][C]);  // valor + tab
    }
    printf("\n");   // nova linha ao fim da linha
  }
}
```

Saída na tela:

7	2	9
4	5	1
8	6	3

O \t separa as colunas; o **\n** pula para a próxima linha.

somarMatriz: A + B → C (com teste de mesa)

Somamos posição a posição: C[L][C] = A[L][C] + B[L][C]. Mesmos dois for.

somar.c

```
void somarMatriz(int A[LINHAS][COLUNAS],
                 int B[LINHAS][COLUNAS],
                 int R[LINHAS][COLUNAS]) {
  int L, C;
  for (L = 0; L < LINHAS; L++)
    for (C = 0; C < COLUNAS; C++)
      R[L][C] = A[L][C] + B[L][C];
}
```

No main: `somarMatriz(A, B, C);  // C recebe o resultado`

Chamamos o resultado de R dentro da função só para não confundir com a variável C da coluna.

A				B				C		
1	2	3	+	9	8	7	=	10	10	10
4	5	6		6	5	4		10	10	10
7	8	9		3	2	1		10	10	10

Ex.: C[0][0] = A[0][0] + B[0][0] = 1 + 9 = 10

C = A + B

Juntando tudo no main

O main declara A, B e C, carrega A e B, soma em C e mostra o resultado.

main.c

```c
#include <stdio.h>
#define LINHAS 3
#define COLUNAS 3
// (carregarMatriz, visualizarMatriz e somarMatriz acima)

int main() {
  int A[LINHAS][COLUNAS], B[LINHAS][COLUNAS],
      C[LINHAS][COLUNAS];
  carregarMatriz(A);
  carregarMatriz(B);
  somarMatriz(A, B, C);   // C = A + B
  printf("Matriz resultado:\n");
  visualizarMatriz(C);
  return 0;
}
```

Lê como uma receita

- carregar A
- carregar B
- somar em C
- visualizar C

Repare

cada procedimento faz uma coisa só e todos percorrem a matriz com os dois for. Trocar 3 por outro tamanho? Só mudar o #define.

Exercício + Gabarito

Exercício

Crie uma função somaElementos que recebe uma matriz e devolve a SOMA de todos os seus elementos.

Dica: um acumulador + os dois for (L e C).

Teste de mesa (com a matriz A da capa)

1	2	3
4	5	6
7	8	9

`soma = 1+2+3+4+5+6+7+8+9`

`resultado: 45`

L vai de 0..2, C de 0..2 — visita as 9 posições.

gabarito.c

```c
// Gabarito
int somaElementos(int M[LINHAS][COLUNAS]) {
  int L, C, soma = 0;
  for (L = 0; L < LINHAS; L++)
    for (C = 0; C < COLUNAS; C++)
      soma += M[L][C];
  return soma;
}
```

Chamada: printf("Soma = %d", somaElementos(A));
É função (retorna valor) — por isso usa return.